BIG DATA COURSE · SECTION 3

MongoDB

From concepts to Compass: connect, create, insert, query, update, delete

1 What is MongoDB · 2 SQL vs NoSQL · 3 Connect · 4 CRUD in Compass

Prepared by: Amira Mahmoud

AGENDA

What we will do today

Meet MongoDB

What it is and why we use it

SQL vs NoSQL

Comparison and vocabulary

Connect

Compass and the local server

CRUD

Create · Read · Update · Delete

MEET MONGODB

What is MongoDB?

Document database

Stores data as flexible JSON-like **documents**

Non-tabular

No fixed rows and columns. Groups of documents are **collections**

Related data together

No JOINs to rebuild a record, so reads are fast

ONE DOCUMENT IN THE students COLLECTION

```
{
  name: "Sara Ahmed",
  age: 20,
  courses: ["SQL", "Big Data"],
  address: { city: "Cairo" }
}
```

MEET MONGODB

Why do we use MongoDB?

Flexible schema

Add fields any time. No `ALTER TABLE`

Scales out

Sharding adds servers as data grows

Fast development

Documents look like objects in your code

Rich queries

Filters, indexes, aggregation

MEET MONGODB

Local or cloud?

LOCAL · used in this course

- Install MongoDB Community Server
- Runs on your own computer
- Free and open source

CLOUD · MongoDB Atlas

- Hosted for you
- No server to manage
- We do not use it here

LOCAL CONNECTION STRING

```
mongodb://localhost:27017
```

SQL VS NOSQL

SQL vs NoSQL: two ways to store the same data

SQL · separate tables

students | courses | addresses

To read one student you JOIN 3 tables

MongoDB · one document

Name, courses and address live together

1 read, no JOIN

SQL · 3 tables and 2 JOINs

```
SELECT s.name, c.title, a.city
FROM students s
JOIN courses c ON c.student_id = s.id
JOIN addresses a ON a.student_id = s.id;
```

MONGODB · 1 query

```
db.students.find({ name: "Sara Ahmed" })
```

SQL VS NOSQL

Comparison table

Feature	SQL (e.g. SQL Server)	NoSQL (MongoDB)
Data model	Tables, rows, columns	Collections of documents
Schema	Fixed, defined first	Flexible, can differ per document
Relations	JOINs and foreign keys	Embedded documents or references
Query language	SQL	MQL: `db.students.find()`
Scaling	Mostly vertical (bigger server)	Horizontal (sharding)
Best for	Structured, transactional data	Varied, fast-changing, large data

SQL VS NOSQL

Same ideas, different names

Database	→	**Database**
Table	→	**Collection**
Row	→	**Document**
Column	→	**Field**
Primary key	→	**_id (automatic)**

If you know SQL Server, you already know the concepts.

MEET MONGODB

Anatomy of a document

DOCUMENT

```
{
  _id: ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"),
  name: "Sara Ahmed",
  age: 20,
  gpa: 3.6,
  courses: ["SQL", "Big Data"],
  address: { city: "Cairo" }
}
```

_id

Unique, created automatically (ObjectId)

Array

`courses` is a list of values

Nested object

`address` is a document inside a document

Flexible

Another student may have different fields

SETUP

Setup in 4 steps

1. Install **MongoDB Community Server** (on Windows choose Install as a Service)
2. Install **MongoDB Compass** (the GUI)
3. Check Services: MongoDB Server = Running
4. Open Compass and connect to `mongodb://localhost:27017`

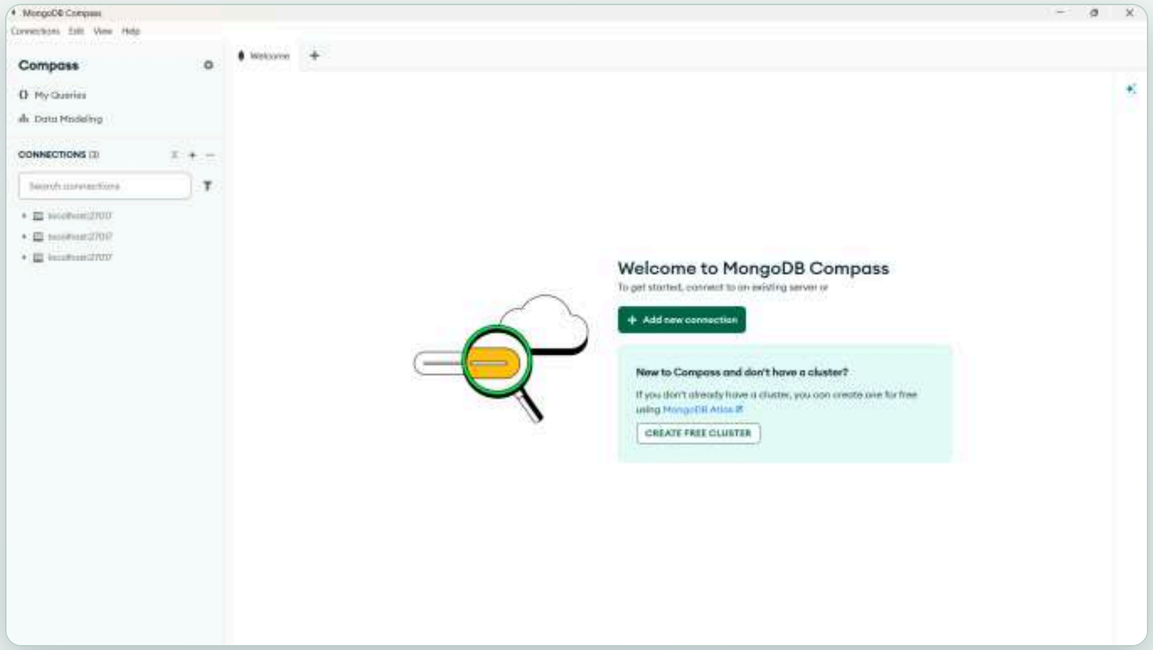
OPTIONAL · check the shell

```
mongosh --version
```

CONNECT

Step 1 · Connect · open Compass

1. Open MongoDB Compass
2. Click **Add new connection**



Compass welcome screen

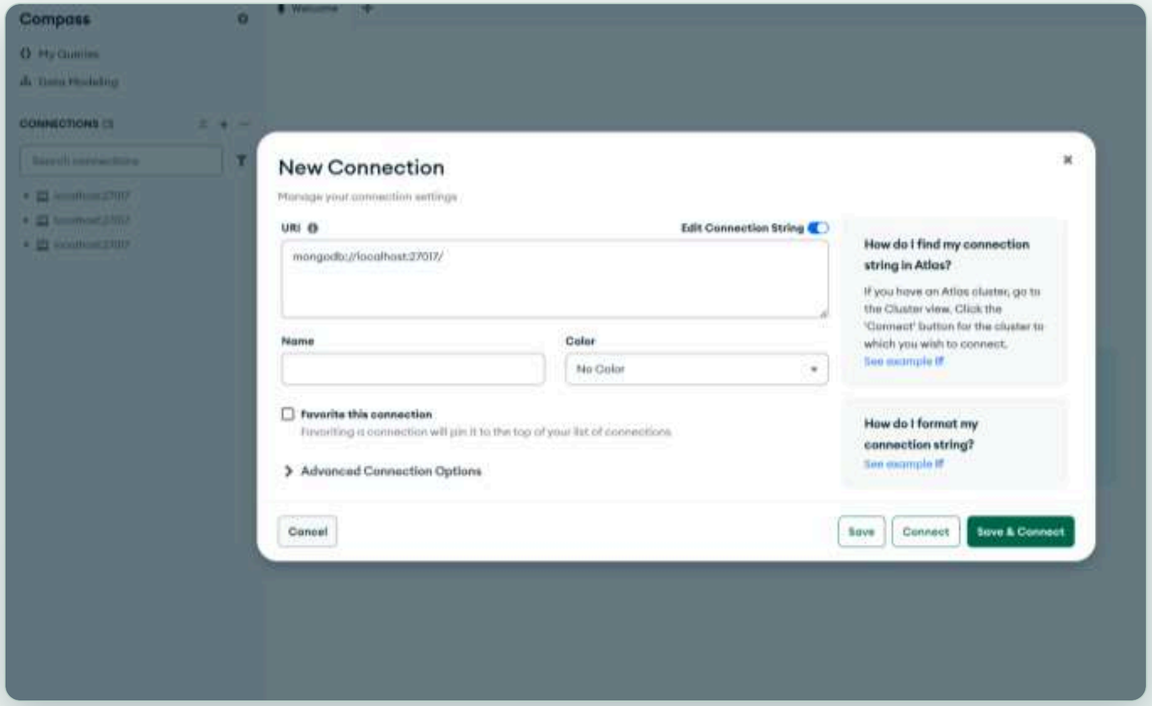
CONNECT

Step 1 · Connect · the form

1. Paste the connection string in the **URI** box
2. Choose Save, Connect or Save & Connect

URI

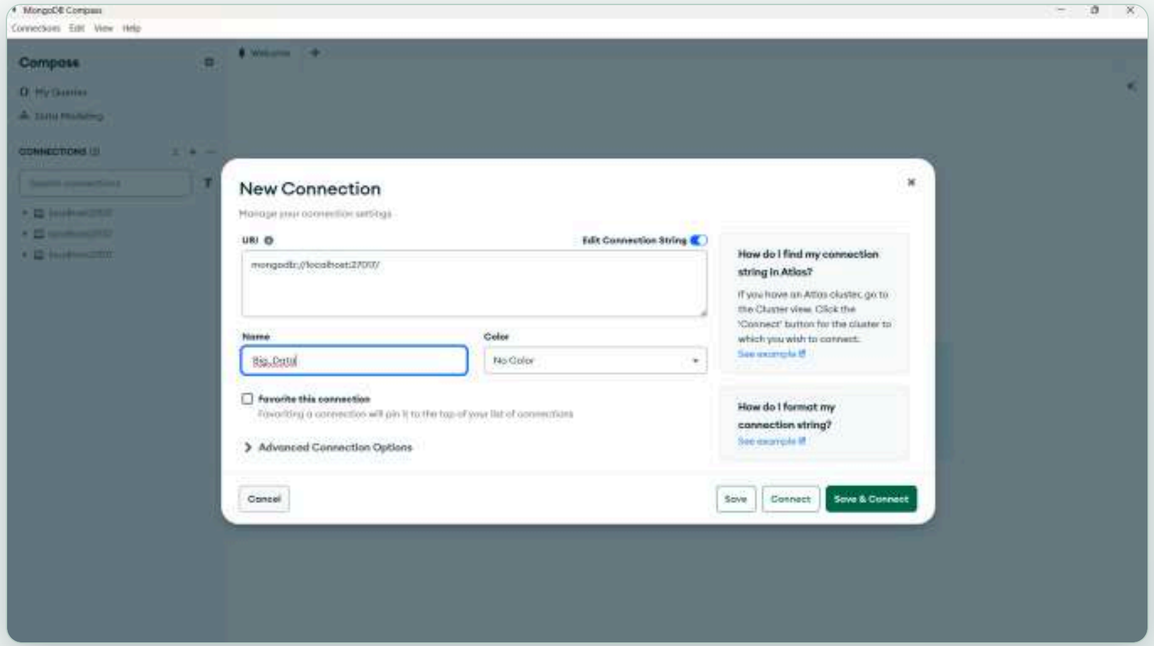
```
mongodb://localhost:27017/
```



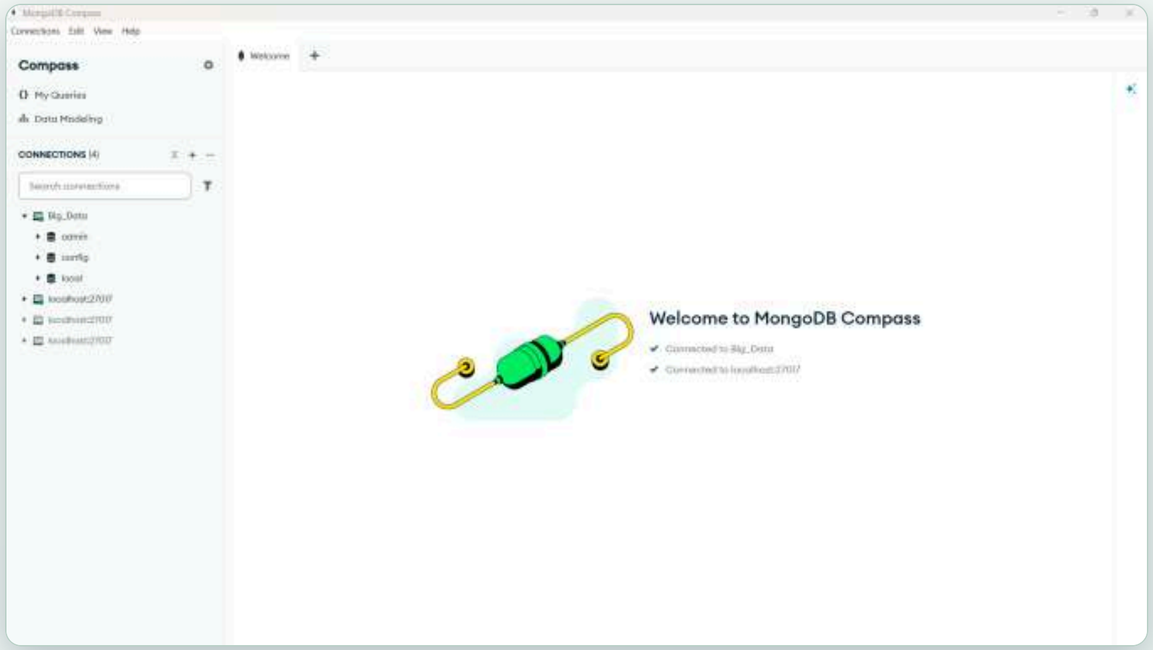
New Connection form

CONNECT

Step 1 · Connect · name it and go



Type a name, then Save & Connect



Connected: admin, config, local

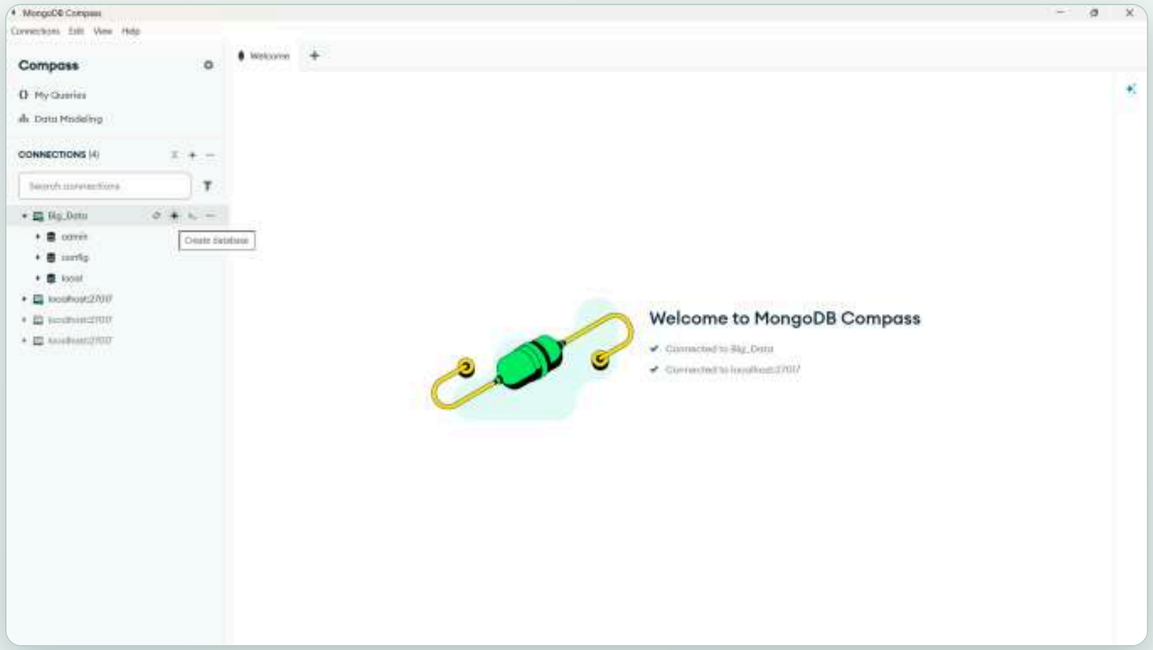
> Type a name such as `Big_Data`, click **Save & Connect**. A green dot appears and the default databases are listed.

CREATE

Step 2 · Create a database

1. Hover over the connection, click the **+** icon
2. Database Name: `MET` · Collection Name: `Students`
3. Keep Time-Series unchecked, click **Create Database**

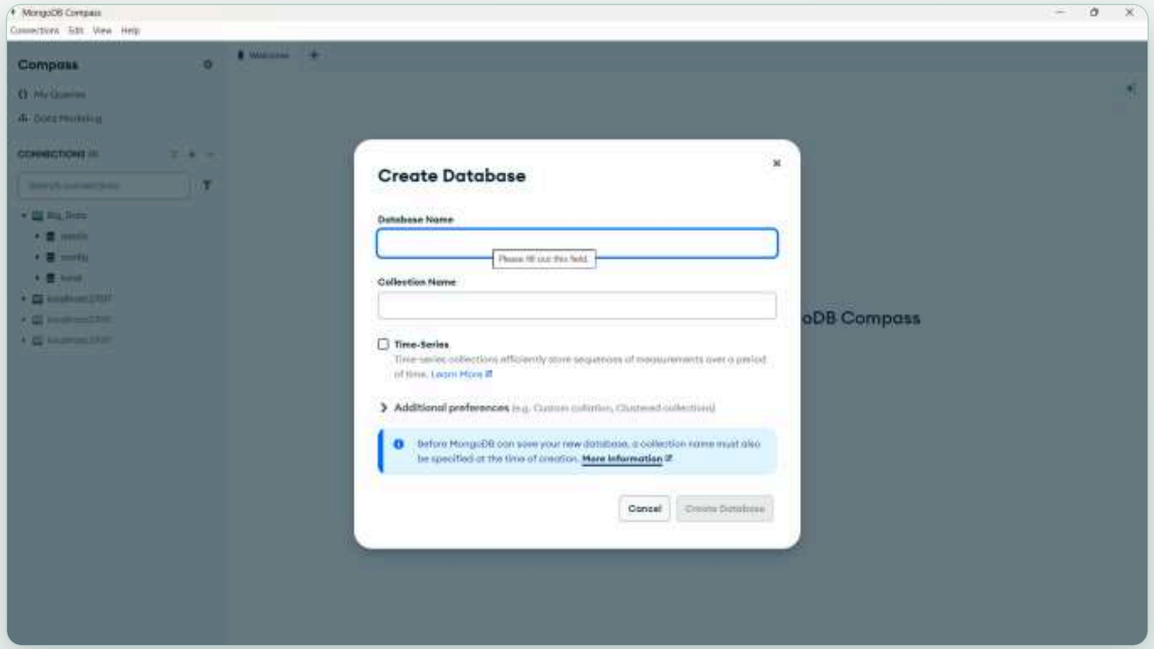
> Both names are required: MongoDB saves a database only together with a collection.



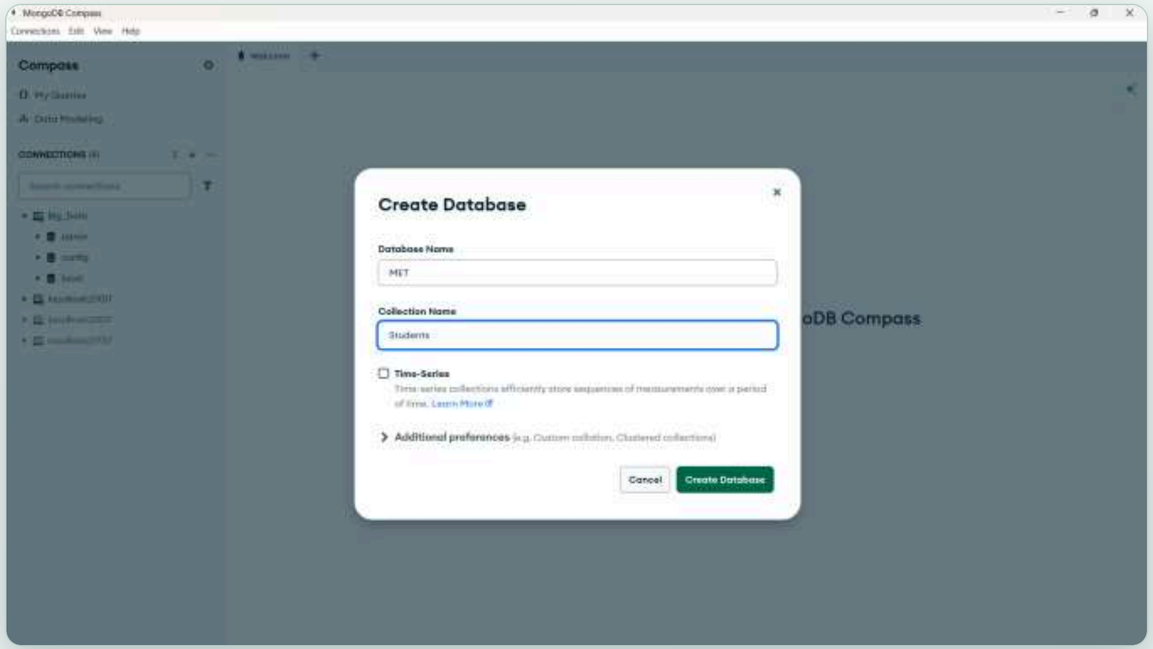
Create database

CREATE

Step 2 · Fill in the names



Empty form: button disabled

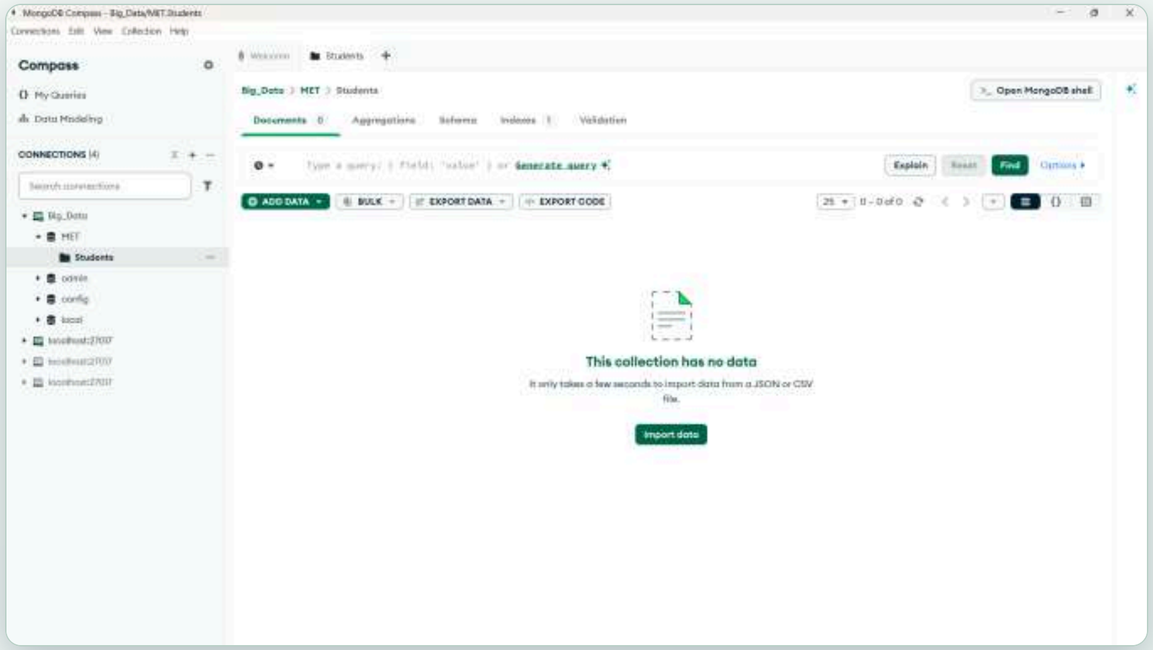


Names filled in

CREATE

The new collection

1. Path shown above the tabs: `Big_Data > MET > Students`
2. Documents counter = 0
3. Use **ADD DATA** to insert, the query bar + **Find** to search



Empty Students collection

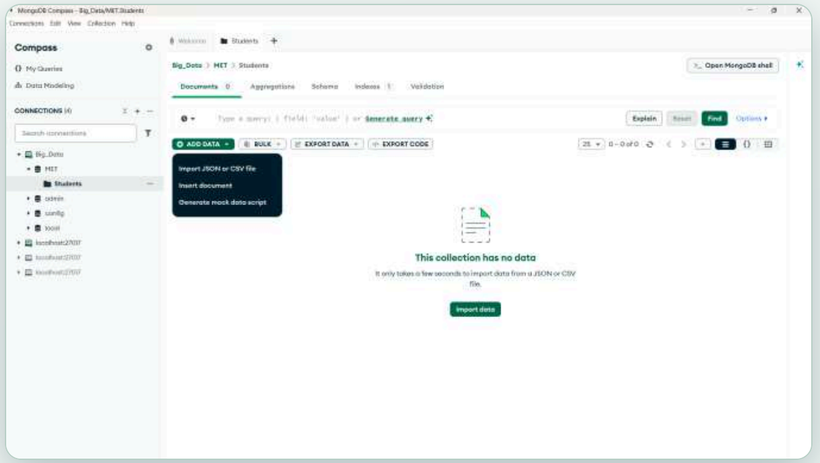
CREATE

Step 3 · Insert one document

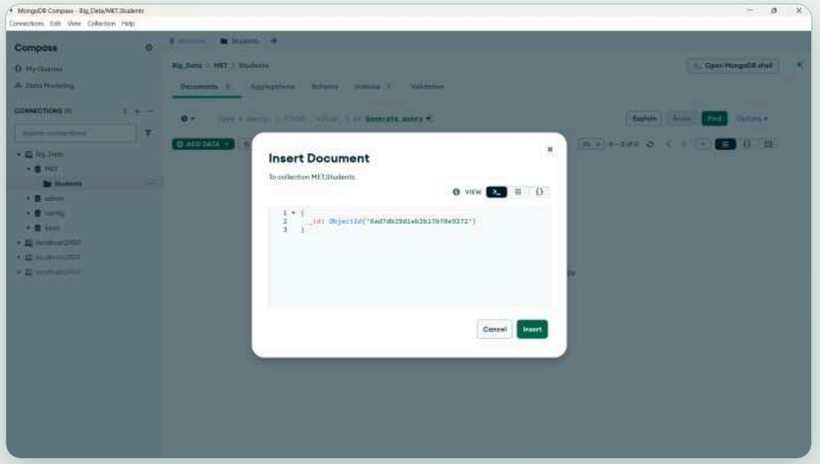
1. Click **ADD DATA**, choose **Insert document**
2. Switch to the JSON view `{ }`
3. Paste the document, click **Insert**

STUDENT 1

```
{
  "name": "Sara Ahmed",
  "age": 20,
  "major": "Computer Science",
  "gpa": 3.6,
  "courses": ["SQL", "DSP", "Big Data", "AI"],
  "address": { "city": "Cairo", "country": "Egypt" }
}
```

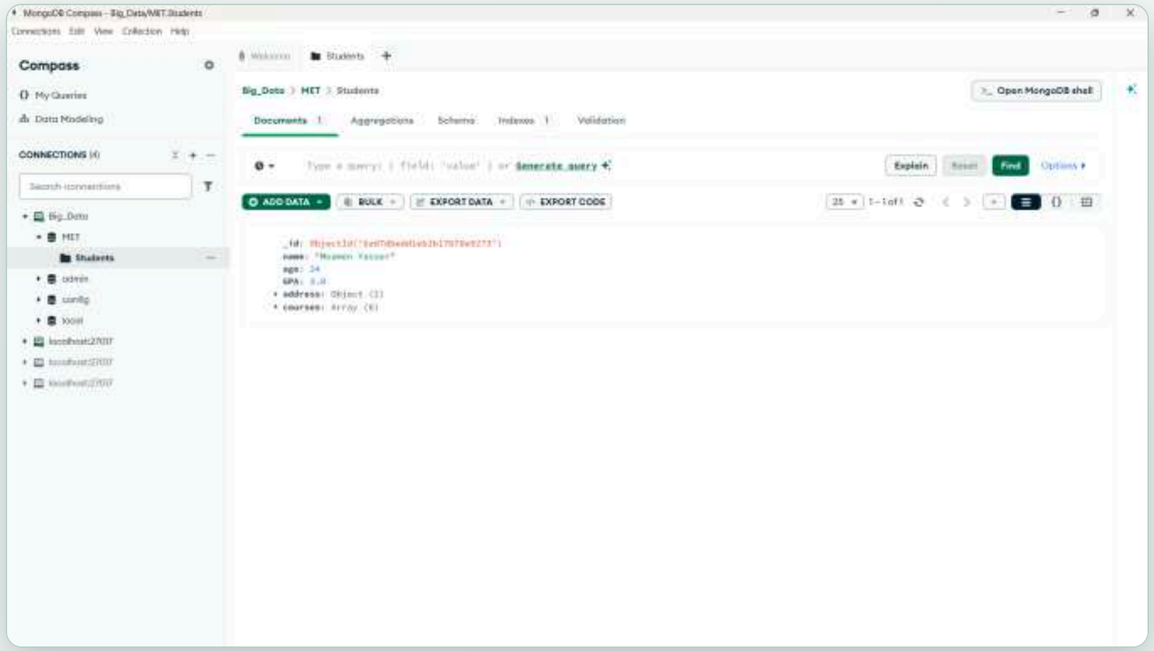


ADD DATA menu

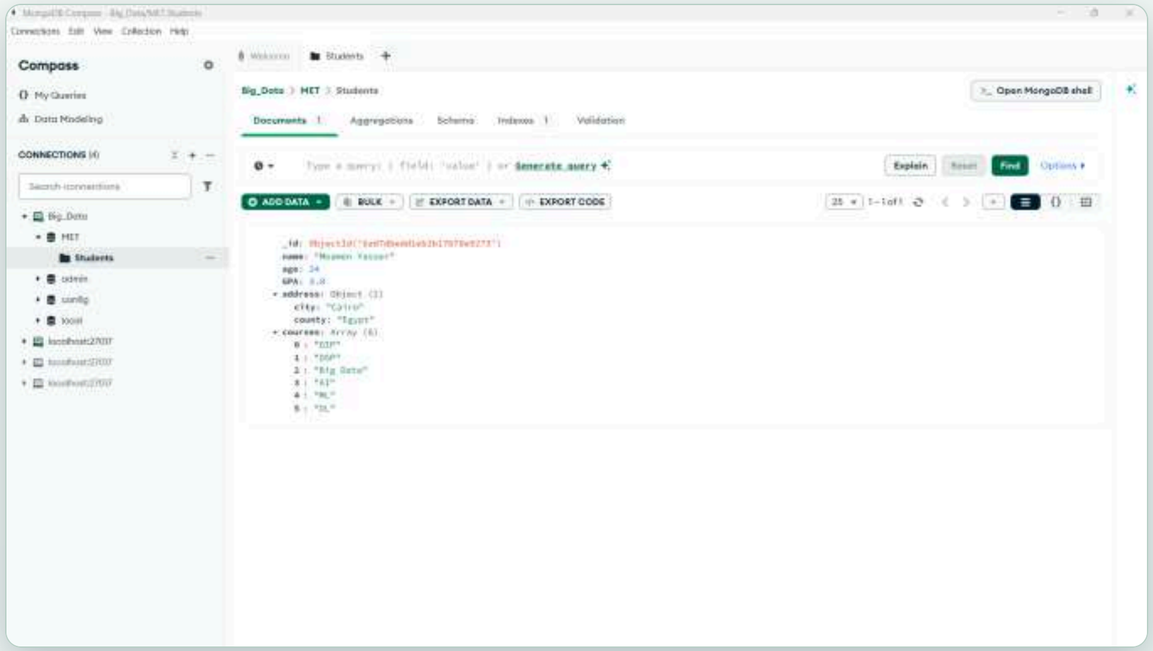


Insert Document dialog

CREATE Step 3 · Insert result



Saved: counter shows 1



address and courses expanded

WHAT YOU SEE

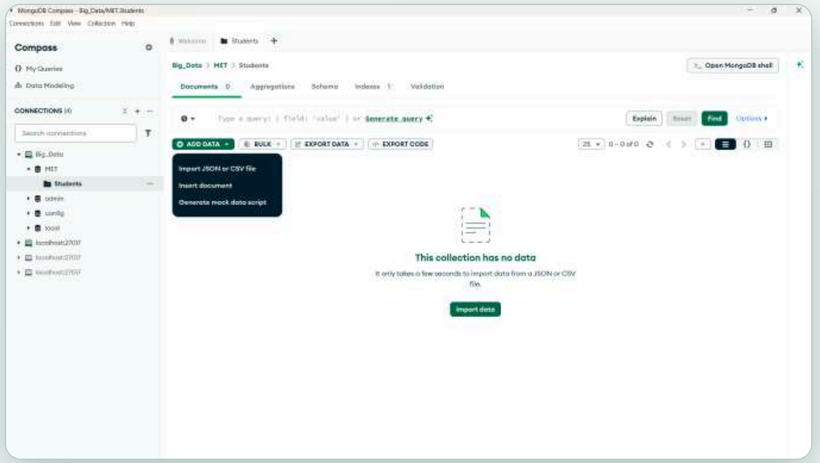
```
Documents 1      1 – 1 of 1
address: Object (2)
courses: Array (4)
```

CREATE

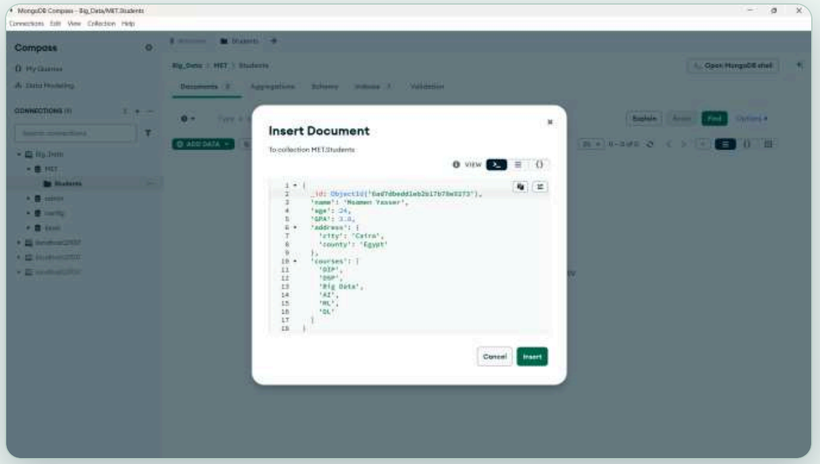
Step 3 · Insert several documents at once

1. Click **ADD DATA**, **Insert document** again
2. Switch to the JSON view `{ }`
3. Delete the default content and paste the arrays from the next two slides, one after the other
4. Click **Insert**

> Several documents go inside [ ] separated by commas.



ADD DATA menu



Paste the documents in JSON view

CREATE Students 2 to 4

ARRAY 1 · 3 STUDENTS · select from [to]

```
[
  { "name": "Ali Hassan", "age": 22, "major": "Data Science", "gpa": 3.2,
    "courses": ["SQL", "Python"], "address": { "city": "Alexandria", "country": "Egypt" } },
  { "name": "Mona Khaled", "age": 21, "major": "Computer Science", "gpa": 3.9,
    "courses": ["Big Data", "AI", "ML"], "address": { "city": "Cairo", "country": "Egypt" } },
  { "name": "Omar Said", "age": 23, "major": "IT", "gpa": 2.8,
    "courses": ["Networks"], "address": { "city": "Giza", "country": "Egypt" } }
]
```

RESULT

```
Documents 4      1 – 4 of 4
```

Use the JSON view { } in the Insert Document dialog. Select the whole block, from the first [to the last].

CREATE Students 5 to 8

ARRAY 2 · 4 STUDENTS · select from [to]

```
[
  { "name": "Nour Adel", "age": 20, "major": "IT", "gpa": 3.4,
    "courses": ["Networks", "SQL"], "address": { "city": "Mansoura", "country": "Egypt" } },
  { "name": "Youssef Tarek", "age": 22, "major": "Data Science", "gpa": 3.7,
    "courses": ["Python", "Big Data", "ML"], "address": { "city": "Cairo", "country": "Egypt" } },
  { "name": "Hana Mostafa", "age": 21, "major": "Computer Science", "gpa": 3.1,
    "courses": ["DSP", "AI"], "address": { "city": "Alexandria", "country": "Egypt" } },
  { "name": "Karim Fathy", "age": 24, "major": "IT", "gpa": 2.5,
    "courses": ["Networks"], "address": { "city": "Giza", "country": "Egypt" } }
]
```

RESULT

Documents 8 1 – 8 of 8

DATA

The 8 students at a glance

Name	Age	Major	GPA	City
Sara Ahmed	20	Computer Science	3.6	Cairo
Ali Hassan	22	Data Science	3.2	Alexandria
Mona Khaled	21	Computer Science	3.9	Cairo
Omar Said	23	IT	2.8	Giza
Nour Adel	20	IT	3.4	Mansoura
Youssef Tarek	22	Data Science	3.7	Cairo
Hana Mostafa	21	Computer Science	3.1	Alexandria
Karim Fathy	24	IT	2.5	Giza

READ Step 4 · Read · find with a filter

1. Type the filter in the query bar
2. Click **Find**
3. Click **Reset** to see everything again

FILTER

```
{ "major": "Computer Science" }
```

3 DOCUMENTS

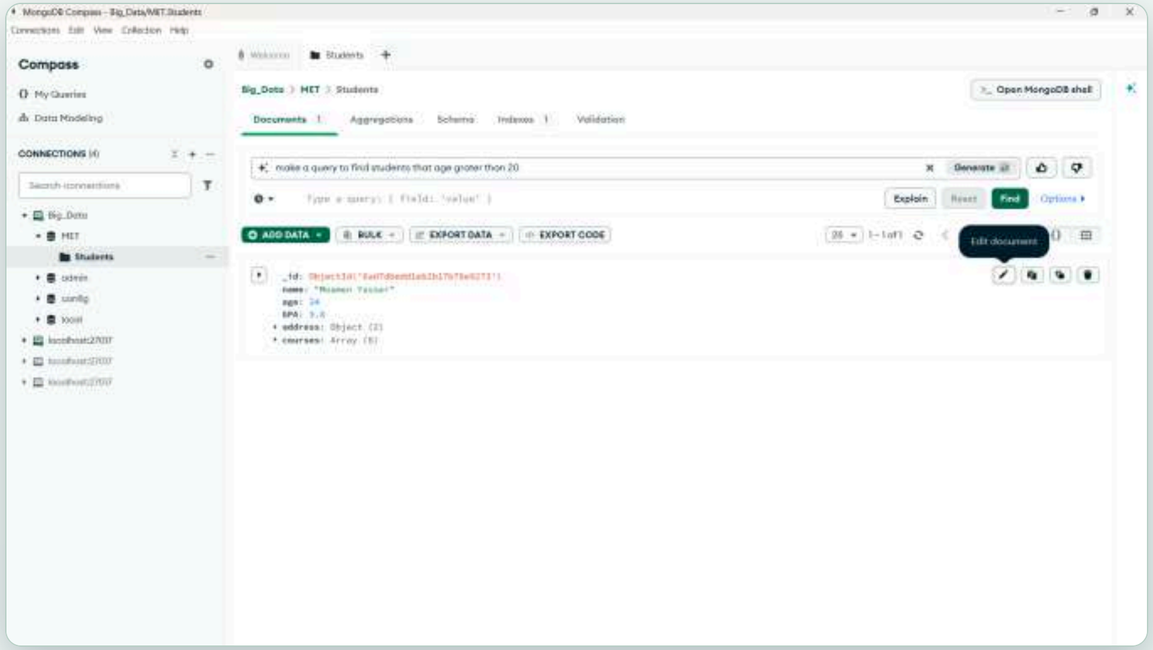
Sara Ahmed · Mona Khaled · Hana Mostafa

NESTED FIELD · quotes required

```
{ "address.city": "Cairo" }
```

3 DOCUMENTS

Sara Ahmed · Mona Khaled · Youssef Tarek



Query bar, Find and document actions

READ Step 4 · Read · comparison operators

GPA 3.5 OR MORE

```
{ "gpa": { "$gte": 3.5 } }
```

3 DOCUMENTS

```
Sara Ahmed 3.6
Mona Khaled 3.9
Youssef Tarek 3.7
```

OLDER THAN 21

```
{ "age": { "$gt": 21 } }
```

4 DOCUMENTS

```
Ali Hassan 22
Omar Said 23
Youssef Tarek 22
Karim Fathy 24
```

Operator	Meaning
`$gt`	greater than
`$gte`	greater than or equal
`$lt`	less than

Matching is case-sensitive and types matter: `"3.6"` is not `3.6`.

READ Step 4 · Read · several conditions

IT STUDENTS WITH GPA BELOW 3

```
{ "major": "IT", "gpa": { "$lt": 3 } }
```

2 DOCUMENTS

```
Omar Said 2.8
Karim Fathy 2.5
```

DATA SCIENCE STUDENTS

```
{ "major": "Data Science" }
```

2 DOCUMENTS

```
Ali Hassan 3.2
Youssef Tarek 3.7
```

> When you write several fields in one filter, a document must match all of them.

READ Step 4 · Read · sort, project, limit

OPTIONS · PROJECT

```
{ "name": 1, "gpa": 1, "_id": 0 }
```

OPTIONS · SORT (-1 = high to low)

```
{ "gpa": -1 }
```

OPTIONS · LIMIT

```
3
```

TOP 3 BY GPA

```
{ name: "Mona Khaled",   gpa: 3.9 }
{ name: "Youssef Tarek", gpa: 3.7 }
{ name: "Sara Ahmed",    gpa: 3.6 }
```

UPDATE Step 5 · Update · change a value

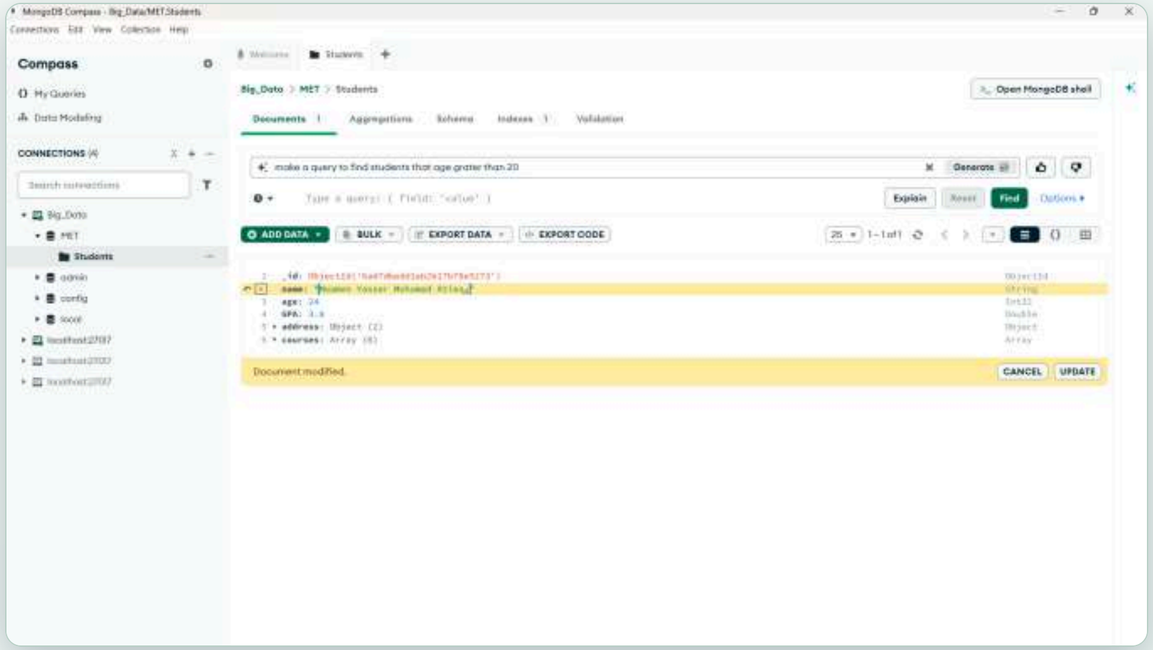
1. Find the document
2. Hover and click the **pencil** (Edit document)
3. Change the value; a yellow bar says Document modified
4. Click **UPDATE** (or CANCEL)

FIND

```
{ "name": "Omar Said" }
```

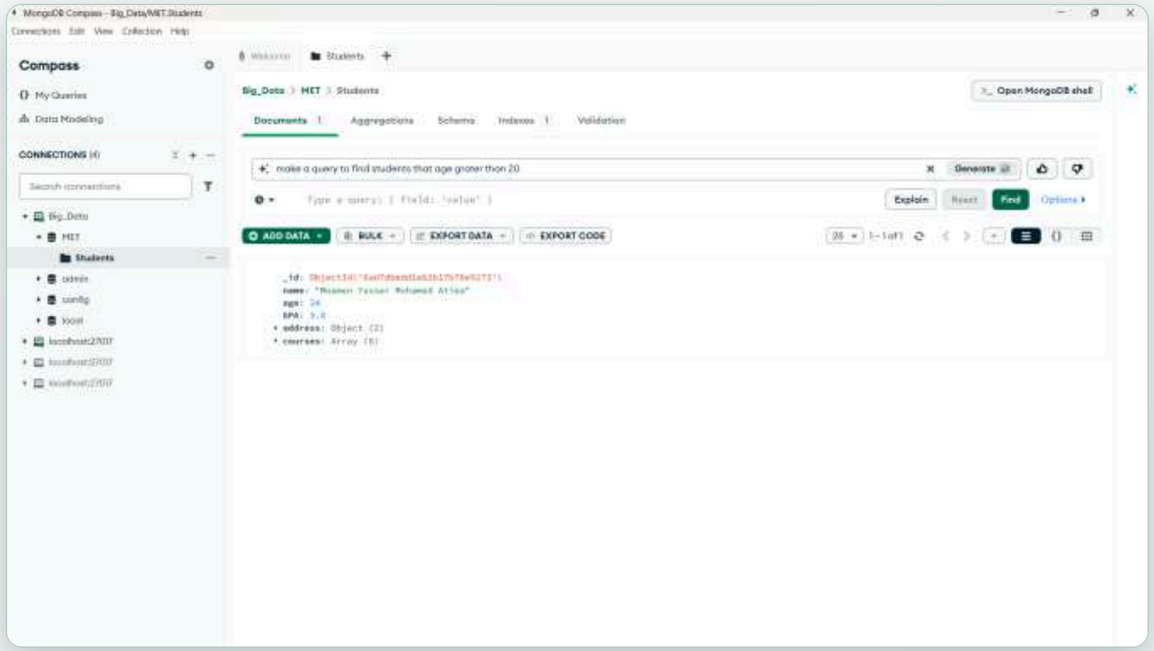
CHANGE

```
gpa: 2.8  →  gpa: 3.0
```



Edit mode with yellow bar

UPDATE Step 5 · Update · add a field and an array item



Update saved: other fields unchanged

NEW FIELD · pick the right type

```
graduated: false   (Type: Boolean)
```

NEW ITEM IN courses (hover last item, click +)

```
"Cloud"
```

RESULT

```
graduated: false
courses: ["Networks", "Cloud"]
```

DELETE Step 6 · Delete a document

1. Find it
2. Hover and click the **trash** icon
3. A red bar says Document flagged for deletion
4. Click **DELETE** to confirm

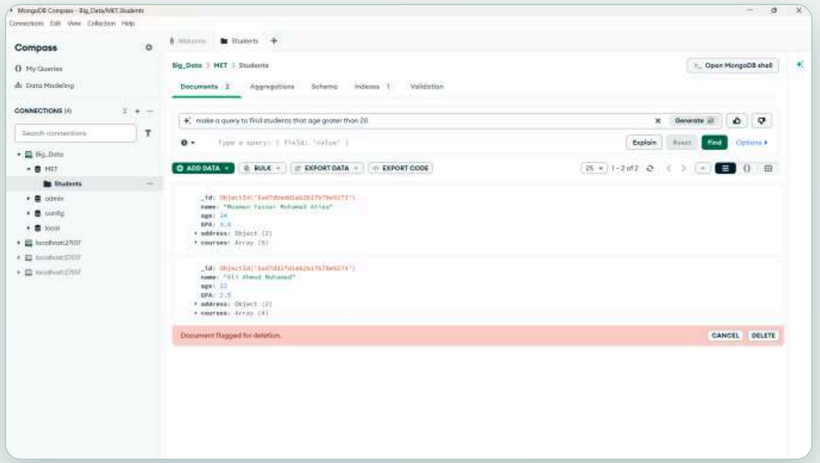
FIND

```
{ "name": "Karim Fathy" }
```

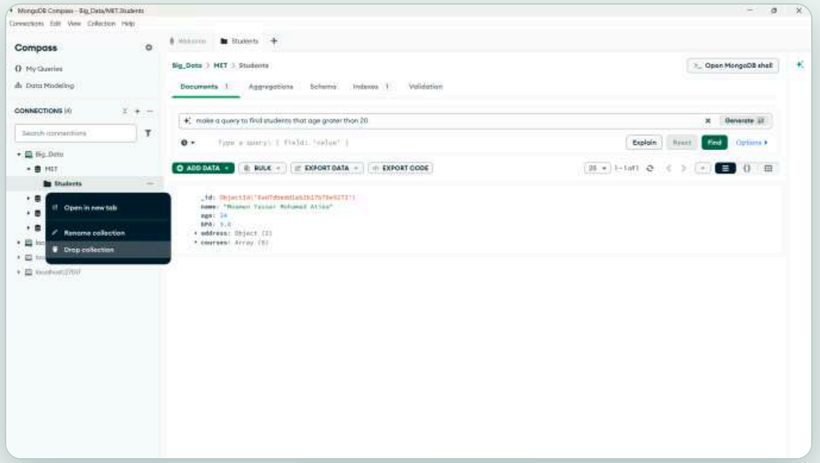
RESULT

```
Documents 7      1 – 7 of 7
```

> There is no undo.



Delete confirmation



Collection menu: rename, drop

RECAP

CRUD cheat sheet

	SQL	In Compass	Code
Create	INSERT	ADD DATA → Insert document	paste JSON
Read	SELECT	Filter bar + Find	`{ "gpa": { "$gte": 3.5 } }`
Update	UPDATE	Pencil → UPDATE	edit values
Delete	DELETE	Trash → DELETE	confirm

RECAP

Key takeaways

Documents
JSON-like, stored in collections

_id
Unique, created automatically

Compass
A client: the server must be running

CRUD
ADD DATA · Find · pencil · trash

Next: CRUD with mongosh, then MongoDB from Node.js.

BIG DATA COURSE · SECTION 3

Thank you

Prepared by: Amira Mahmoud